

DroneLogix — Two-Site Drone Delivery Marketplace

Structure Guide (built on your SmartLogix PDSA design)

This adapts your existing 5-module SmartLogix design (Dijkstra, Knapsack, Prim's, Decision Tree, Simulated Annealing) to a **drone-delivery marketplace** split across **two Next.js sites** sharing one Supabase backend.

1. Overall Concept

Site A — Marketplace	Public ecommerce site for buyers. Browse, buy, track drone delivery.
Site B — Seller & Admin Portal	Sellers list products in their warehouse; Admins manage drones, warehouses, network, and the landing/marketing site.
Shared backend	One Supabase project. Both sites read/write the same tables, separated by Row Level Security (RLS) per role.
Physical constraint driving everything	Every drone has a max payload of 85kg and a max cargo bay volume . This constraint is what turns your Knapsack module into a <i>2D/3D constrained knapsack</i> instead of a plain 1D one — a nice complexity upgrade for your report.

You keep all 5 algorithm modules from your original doc — you're just re-skinning what they operate on:

Task	Original (SmartLogix)	Your version (DroneLogix)
1. Route Optimization (Dijkstra)	Truck route between hubs	Drone flight path between warehouse and delivery zone, edges = flight corridors, weights = distance/no-fly-zone penalty
2. Resource Allocation (Knapsack)	Load cargo onto one vehicle by weight	Load orders onto one drone by weight ($\leq 85\text{kg}$) and volume (cargo bay size) — this is your intractable-adjacent constraint, good talking point
3. Network Analysis (Prim's MST)	Minimum-cost hub infrastructure	Minimum-cost charging-station / warehouse network so every seller's warehouse has a cheapest-cost link into the drone network
4. Decision Module (Decision Tree)	Supplier risk classification	Delivery feasibility / drone-assignment classification — e.g. classify an order as Drone-Deliverable / Needs-Split / Rejected based on weight, size, distance, weather flag
5. Optimization Module (Simulated Annealing)	Multi-stop VRP/TSP for one vehicle	Multi-drone delivery batching — one drone doing 2-4 lightweight stops in one flight before returning to base, closed-loop tour per drone

2. Site A — Marketplace (Buyer-facing)

Pages

Home — featured products, categories, "how drone delivery works" banner

Category / Browse — filter by category, price, seller, delivery-zone availability

Search results

Product Detail Page

Images, price, stock, seller name

Weight & dimensions shown to buyer (so oversized/heavy items can visibly warn "may require split delivery")

Delivery estimate (calls Route Optimization module for distance/ETA from nearest warehouse with stock)

Cart

Checkout

Delivery address → captures lat/long (needed for Dijkstra graph + drone range check)

Address must fall inside a warehouse's drone range, or checkout blocks/upsells alternate fulfillment

Order Confirmation

Order Tracking (live)

Map showing drone's current position (Supabase Realtime)

Status: Processing → Allocated to Drone → In Flight → Delivered

Order History

Account / Profile

Support / Help

Buyer-facing product fields (subset of full product record)

Name, price, images, short description, stock available, seller name, estimated delivery time, weight class badge (e.g. "Standard", "Heavy — surcharge"), category.

3. Site B — Seller & Admin Portal

3.1 Seller Dashboard

Login/Register (seller application → admin approval step)

Dashboard home — sales summary, pending orders, low stock alerts

Product Management

Add/Edit Product (full field list below)

Bulk stock update

Inventory by Warehouse — sellers may store stock across multiple company warehouses

Orders Received — order list, status, packed/ready-for-pickup toggle

Earnings / Payouts

Seller Profile / Store Settings

3.2 Admin Panel

Admin Dashboard — system-wide KPIs, active drones, orders in flight

Seller Management — approve/reject/suspend sellers

Warehouse Management

Add/edit warehouse (location, capacity, number of drone docks)

Drone Fleet Management

Add/edit drone (see fields below)

Drone status board (Available / In-Flight / Charging / Maintenance)

Manual override / reassignment

Network Analysis Dashboard — runs Prim's MST across warehouses/charging stations, shows infrastructure cost

Route Optimization Console — test Dijkstra/A*/Bellman-Ford between any two points, compare results

Delivery Batching Console — trigger Simulated Annealing for the day's queued orders, compare vs Genetic Algorithm / exact DP for small n

Decision Module Console — view/retrain the delivery-feasibility Decision Tree, audit classification rules

Benchmark / Evaluation Dashboard — the `benchmark_logs` charts required for your report (Chapters 7–9 in the original doc)

Landing Page CMS — edit marketing copy/images for the public landing page

User & Role Management

4. Product Fields (Supabase `products` table)

Field	Type	Notes
id	uuid	PK
seller_id	uuid	FK → profiles
warehouse_id	uuid	FK → warehouses (which warehouse holds stock)
name	text	
description	text	
category	text	
price	numeric	
stock_qty	int	
weight_kg	numeric	hard max 85kg — enforced at insert (product alone must be ≤85kg since one drone can't split a single item)
length_cm / width_cm / height_cm	numeric	dimensions, checked against drone cargo bay envelope
volume_cm3	numeric (generated)	length × width × height, used by the constrained-knapsack allocation
fragile	boolean	affects drone flight profile (slower/steadier)
images	text[] / storage refs	
status	enum	draft / active / out_of_stock / suspended
created_at	timestampz	

Validation rule to enforce in the seller "Add Product" form: reject if `weight_kg > 85` or if any dimension exceeds the largest drone's cargo bay — with a clear error message, since that item is physically undeliverable by your fleet.

5. Drone Fields (new table: `drones`)

Field	Type	Notes
id	uuid	PK

Field	Type	Notes
drone_code	text	e.g. "DRN-014"
model	text	
max_payload_kg	numeric	e.g. 85 (can vary if you later add drone tiers)
cargo_bay_length_cm / width_cm / height_cm	numeric	defines max item/order envelope
max_range_km	numeric	used by Dijkstra to prune unreachable edges
battery_capacity_pct	numeric	current charge
speed_kmh	numeric	for ETA calc
status	enum	available / in_flight / charging / maintenance
home_warehouse_id	uuid	FK → warehouses
current_lat / current_lng	numeric	live position, updated via Realtime

Warehouse fields (`warehouses`)

id, name, seller-facing flag, latitude, longitude, capacity, **drone_dock_count**, charging_station boolean.

Order → Drone assignment table (`drone_assignments`)

id, order_id(s) (an assignment can cover multiple orders batched onto one flight), drone_id, route_id, total_weight_kg, total_volume_cm3, status, departed_at, delivered_at.

6. How the 5 Algorithm Modules Actually Get Used Here

Task 1 — Route Optimization (Dijkstra): Graph nodes = warehouses + delivery zones + optional relay/charging stations. Edge weight = flight distance (and optionally a no-fly-zone or wind penalty). Run when checkout confirms a delivery address, to compute distance/ETA and confirm the address is within range of a warehouse that has stock.

Task 2 — Resource Allocation (Constrained Knapsack): Instead of 1D (weight only) like the original truck version, this becomes a **2-constraint knapsack**: maximize value of orders loaded onto one drone subject to `Σweight ≤ 85kg` AND `Σvolume ≤ cargo_bay_volume`. Still solvable with a DP table, just indexed on both constraints (or volume as a secondary greedy/feasibility filter after weight-based DP) — a good "extension" to discuss in your report as harder than the base 0/1 Knapsack.

Task 3 — Network Analysis (Prim's MST): Nodes = warehouses/charging stations, edges = cost to build a direct flight corridor/charging link. Used by Admin to justify where to build new charging stations at minimum infrastructure cost as the seller network grows.

Task 4 — Decision Module (Decision Tree): Reframe from "supplier risk" to "**delivery feasibility classification**": inputs = weight, volume, distance to nearest capable drone, battery/range margin, weather flag → output = Drone-Deliverable / Requires Split Order / Reject. Transparent rules matter here because it's a real go/no-go decision shown to sellers and ops staff.

Task 5 — Optimization Module (Simulated Annealing): Batches multiple *lightweight* orders going to nearby addresses onto a single drone flight as a closed-loop tour (depot → stop1 → stop2 → depot), same circular-linked-list model as your original doc, just applied per-drone per-batch instead of per-truck per-day.

7. End-to-End Flow

Seller adds a product (weight/dimensions validated against fleet limits) to their warehouse's stock.

Buyer orders on Site A; checkout captures delivery coordinates.

System runs **Dijkstra** to confirm reachability + ETA from the warehouse holding stock.

System runs the **Decision Tree** to classify feasibility (single-drone deliverable vs needs splitting vs rejected).

Orders queue up; when enough orders are pending for one warehouse, **constrained Knapsack** selects which combination of orders best fills one drone's weight+volume capacity.

If multiple stops fit geographically, **Simulated Annealing** sequences them into one closed-loop flight.

Drone departs — `drone_assignments` status updates flow via Supabase Realtime to buyer tracking page, seller order list, and admin dashboard.

Periodically, Admin runs **Prim's MST** to review/expand the warehouse/charging-station network.

Every run of every algorithm logs to `benchmark_logs` for your coursework's evaluation charts.

8. What Changes vs. Your Original SmartLogix Doc (summary)

One Next.js app → **two** Next.js apps (Marketplace, Seller+Admin Portal), same Supabase project.

`vehicles` table → `drones` table, with payload **and** dimension limits instead of just weight.

Knapsack becomes a **two-constraint** allocation problem (weight + volume).

Decision Tree's business meaning shifts from "supplier risk" to "delivery feasibility."

Add `warehouses.drone_dock_count`, `drones`, `drone_assignments` tables; extend `products` with weight/dimension fields and a max-85kg check.

Route graph nodes are physical flight points, not just road-connected hubs — worth a line in your report about why Dijkstra still applies (non-negative edge weights, sparse graph of known corridors).

Want me to turn the DB tables above into an actual Supabase SQL schema, or draft the wireframe/page list for one of the two sites (e.g. the seller's "Add Product" form) next?